THE PLAYGROUND

Product specification & master build prompt

A multifunctional, browser-based collection of tactile toys, audio tools, visual experiments and arcade mini-games.

Target: GitHub Pages · Static hosting · Offline-capable PWA · Desktop and mobile browser

Build philosophy: One cohesive product, modular experiences, responsive controls, honest capability fallbacks, and polished interaction design.

This document is intended to be supplied to an AI coding agent as the product brief. It defines the behaviour, interface, technical boundaries and acceptance criteria for each experience.

1. Product vision

Create a single web app called **THE PLAYGROUND** (working title): a multifunctional interactive playground containing eleven experiences. It should feel like a carefully designed piece of software rather than a collection of unrelated demos. The user should be able to move between experiences from a consistent home screen, while each experience gets a purpose-built interface.

The visual identity should combine a premium dark instrument-panel aesthetic with selected retro/pixel-art treatments. The Garage should be distinctly pixel/arcade-inspired; the audio and fidget tools should feel tactile and physical; the art studio should feel creative; games should be instantly understandable.

Primary goals: satisfying interactions, low-friction use, fast loading, responsive layout, keyboard and touch support, local persistence, and no mandatory account or backend.

2. Platform and technical constraints

• **Hosting:** GitHub Pages static hosting. The production build must work from a repository subpath (configure the correct base path; do not assume the app is hosted at domain root).

• **Suggested stack:** Vite + TypeScript + React, or a similarly modular static front-end stack. Use CSS variables/design tokens. Use Canvas 2D for particle/physics scenes and Web Audio API for audio processing.

• **No backend dependency:** no login, server database, paid API, or API key. Save preferences, scores, presets and recent settings in localStorage or IndexedDB.

• **PWA:** web app manifest, service worker, install instructions, offline app shell and graceful offline behaviour. Do not claim every feature works offline if it relies on browser/device capabilities.

• **Mobile and desktop:** responsive from narrow phones to desktop. Touch targets should be generous; desktop controls should support mouse and keyboard.

• **Performance:** target smooth animation; use requestAnimationFrame, pause inactive scenes, cap particle counts on weaker devices, clean up audio nodes/listeners, and avoid unnecessary continuous rendering.

• **Accessibility:** visible focus states, semantic controls, labels, keyboard alternatives, reduced-motion option, sufficient contrast, and non-colour-only status cues.

Important browser limitations: The Vibration API is not supported on every browser/device and often does not work on desktop. Audio output equalization can only process audio routed through the app's own audio graph; a normal website cannot globally equalize all system audio. Microphone access requires permission and a secure context. Image processing should happen locally in the browser.

3. Global information architecture

• **Home / Launchpad:** visually rich grid of eleven experience cards, each with title, short description, category and a clear launch action. Show recently used experiences and pinned favourites.

• **Persistent top bar:** app mark, Home, search/filter experiences, sound toggle, haptics toggle, settings and a compact status indicator for device capability.

• **Experience shell:** each module has a consistent title, back-to-home action, help/controls panel, reset action and fullscreen option where useful.

• **Settings:** master volume, per-experience sound preference, haptics preference, reduced motion, visual quality (Auto/High/Low), theme accent, and data reset/export.

• **Persistence:** save user settings, art presets, game best scores and last-used state locally. Provide a clear reset option.

4. Shared interaction and feedback system

• **Audio:** provide a master mute and per-module sound toggle. Never autoplay sound before a user gesture. Keep sound effects short, optional and volume-controlled.

• **Haptics:** use navigator.vibrate only after a user gesture and only when supported. Add a capability check and a no-op fallback. Do not imply that haptics work on all devices. Offer intensity/pattern presets where possible.

• **Physicality:** knobs should rotate or respond to drag, sliders should have tactile visual feedback, buttons should have pressed states, and controls should provide immediate visual/audio/haptic feedback.

• **Input:** support pointer, touch and keyboard where appropriate. Prevent accidental page scrolling only inside active game canvases, not across the whole site.

• **Feedback settings:** sound and haptics should be independently switchable. Respect reduced-motion and browser accessibility settings.

5. Experience specifications

A. Audio Visualizer & Output Lab

Purpose: an audio playground for audio generated inside the app or audio files the user opens. It must not claim to modify all system-wide audio.

Main layout

• Large central visualizer: selectable waveform, spectrum bars, circular spectrum, oscilloscope, mirrored bars and particle-reactive modes.

• A tactile control deck below or beside the visualizer with rotary knobs for Bass, Treble, Mid, Gain/Amplitude, Balance and visualizer sensitivity.

• Preset strip: Flat, Warm, Bright, Bass Boost, Vocal and user-saved presets. Include a reset-to-flat control.

• Audio source area: play a built-in demo tone/loop or load a local audio file. Show play/pause, seek, elapsed time and volume.

Audio behaviour and controls

• Use an HTMLAudioElement or generated oscillator as the source, connected to an AudioContext graph. Use BiquadFilterNodes for bass/mid/treble shaping and GainNode for output level.

• Bass knob controls a low-shelf filter; Treble controls a high-shelf filter; Mid controls a peaking filter; Gain controls output gain. Define sensible bounded ranges and display values.

• The visualizer reads analyser data from the same graph. Amplitude/sensitivity should affect visual response, not secretly alter loudness.

• Knobs must support drag, touch and keyboard adjustment; display numeric values and provide double-click/tap reset.

• Use generated sound only as a demo/test source; do not bundle copyrighted music.

Acceptance: loading a local audio file and changing EQ must audibly affect that file within the app; visualizer must react to the processed signal; mute, reset, presets and keyboard controls must work.

B. Garage Simulator

Purpose: a retro pixel-art driving playground with two distinct modes: an arcade lane-dodging game and a stationary/controlled car sound test bench.

Garage home and car selection

• Show a small garage/selection screen with at least three fictional, visually distinct cars (for example: compact hatch, rally coupe, classic muscle). Use original names and generated/synthesized sound design rather than copyrighted brand

assets.

• Each car has a pixel-art sprite, colour variants, basic handling profile, engine pitch character and optional acceleration profile.

• Provide a preview/rev control and a clear mode selector: **Arcade Drive** and **Engine Lab**.

Mode 1 — Arcade Drive

• Portrait-friendly vertical road viewed from behind/above the player's car. The road scrolls downward to create forward motion. Player vehicle remains near the lower centre.

• Three or more lanes. Swipe left/right, tap lane arrows, or use keyboard arrows/A-D to change lanes. Smooth lane transitions; clamp the player to valid lanes.

• Avoid traffic, cones, barriers and occasional moving obstacles. Collision ends the run or removes health, depending on selected difficulty.

• Accelerate and brake controls (buttons, keyboard or press-and-hold) change game speed. If speed/gear simulation is simplified, omit RPM and gear controls in this mode rather than displaying fake telemetry.

• Distance, score, current speed, best score, pause and restart. Difficulty gradually increases obstacle frequency and speed.

• Pixel-art road markings, dithering, limited palette, sprite animation, parallax scenery and optional CRT scanline treatment. Keep gameplay readable; effects must not obscure hazards.

• Car wheel animation should rotate in proportion to speed where visible. Add subtle wind streaks/road motion at higher speeds.

Mode 2 — Engine Lab

• Use a wide central instrument layout: left RPM gauge, centre car display/sprite, right speed gauge. The central car must be visually prominent and aligned between the two gauges.

• Show gear state, clutch state, throttle input and engine status. Controls: ignition/start, throttle/rev, clutch, gear up/down or a clickable gear selector, brake and stop engine.

• Model a simplified manual transmission state machine: engine can rev while clutch is pressed; gear changes are allowed when clutch is pressed; releasing clutch while in gear couples engine speed to vehicle speed in a simplified way. Prevent impossible or confusing states with helpful feedback.

• The vehicle can be stationary while revving. Speed should only rise when the simplified drivetrain state and throttle permit it. Include neutral and reverse only if they can be implemented clearly.

• RPM needle responds smoothly to throttle and gear load. Speed gauge responds to simulated vehicle speed. Add redline zone and engine warning feedback.

• Wheel rotation and wind animation should correspond to simulated speed. When stationary, wheels stop and wind effect is absent.

• Car sound should change with RPM: use layered generated engine tones/noise or original audio samples, with pitch, harmonics and intensity changing smoothly. Add gear-shift, clutch, starter and idle sounds.

• Include a sound test panel to compare cars and replay idle, rev, acceleration, shift and deceleration behaviours. Add volume and mute.

Important: this is a playful simulation, not a realistic vehicle training tool. Label the drivetrain as simplified. Prioritize responsive controls and satisfying feedback over mechanically exact physics.

C. Generative Art Studio

• Provide a blank canvas mode with geometric generators: Voronoi-like cells, polygon mosaic, low-poly facets, geometric grids, radial patterns and procedural shapes.

• Allow user-defined palettes: colour picker, add/remove swatches, random palette, saved palettes and optional image-derived palette.

• Photo-to-art flow: select or drag in a local image; process locally in browser; offer geometric segmentation styles and adjustable detail/complexity, edge emphasis, colour quantization and facet size.

• Allow clicking a region/facet and filling it with a chosen colour; include paint bucket, eyedropper, undo/redo, reset and zoom/pan.

• Export final art as PNG. Keep original uploaded image local; do not upload it anywhere. Show a clear notice that large images may take longer.

• Provide a before/after toggle and a non-destructive editing approach where feasible.

Acceptance: user can upload a photo, generate a geometric interpretation, recolour individual regions, undo edits and export a PNG without a server.

D. Atmosphere — Ambient Sound Mixer

• A mixer of independent ambience layers: rain, thunder, wind, ocean, forest, fireplace, café murmur, fan and white noise. Use original generated loops or properly licensed audio assets.

• Each layer has play/pause and independent volume slider. Provide master volume, mute, fade-in/fade-out and a few curated presets such as Focus, Rainy Window, Night Drive and Forest.

• Timer with optional fade-out; save custom mixes locally.

• Use a visual scene or subtle animated background that responds to active layers. Keep visuals low-motion by default.

• No autoplay. If audio assets are not bundled, clearly explain the limitation rather than showing broken controls.

E. Color Clash

• A fast colour recognition game. A prompt asks for a target colour or colour name while the screen presents coloured tiles or conflicting text/colour combinations.

• Include multiple game modes: match the swatch, identify the odd one out, name-versus-colour challenge and timed survival.

• Scoring rewards correct answers and speed; wrong answers reduce time or streak. Difficulty increases through smaller differences and faster rounds.

• Include 30/60/90-second sessions, best score, accuracy, streak, pause and restart. Use accessible labels and do not rely only on colour for essential instructions.

F. Lights Out

• Grid puzzle where selecting a tile toggles that tile and its orthogonal neighbours. Goal: turn every light off.

• Board sizes: 3×3, 4×4, 5×5 and 6×6. Generate solvable boards by applying a random sequence of valid toggles to an all-off board.

• Show move count, timer, difficulty, hint (optional), undo, reset and new puzzle. Include a move-limit-free relaxed mode.

• Use clear on/off states, smooth transitions and optional click/haptic feedback.

G. Number Merge

• A 2048-style grid puzzle. Swipe or use arrow keys to move tiles. Equal values merge once per move; new low-value tiles spawn in empty cells.

• Modes: classic 4×4, relaxed larger grid, and timed score attack if feasible. Add undo (limited or unlimited in relaxed mode), restart, score, best score and game-over detection.

• Use smooth tile movement/merge animations, readable values and clear contrast. Persist best score locally.

• Make game logic deterministic enough to test; prevent double-merging a tile in one move.

H. Newton's Playground

• A physics sandbox with pendulums, Newton's cradle, bouncing balls, domino chains, springs and gravity wells.

• User can drag objects, release them, adjust gravity, damping, mass and restitution, and trigger reset or slow motion.

• Include preset experiments and an optional free-build mode. Keep simulation stable with bounded time steps and sensible object limits.

• Use soft collision/impact sounds and haptic pulses on supported devices for notable impacts. Add sound/haptic toggles and reduced-motion support.

• Physics should be visually convincing and consistent; it need not be a scientific simulator. Clearly distinguish playful approximations from exact measurements.

I. Digital Kinetic Sand

• A touch-first particle sandbox where grains fall, pile up and flow around obstacles under gravity.

• Tools: pour, scoop/erase, push, rake, wall/shape placement and clear. Include brush size and particle density.

• Materials/palettes: sand, dark sand, glitter-like grains and a few colour choices. Avoid overloading the initial version with too many materials.

• Use Canvas 2D with optimized particle updates, spatial partitioning or simplified cellular simulation. Add quality presets for mobile performance.

• Optional granular pouring and brushing sounds; haptics for supported devices. Include pause, reset and screenshot/export if practical.

J. Fidget Lab

• A collection of tactile digital fidget objects presented as a shelf/grid: classic spinner, tri-spinner, gear spinner, clicker, toggle switches, sliders, rotary dial, pop buttons, rolling bead and ratchet.

• Each object has a distinct interaction model and feedback. Spinners accelerate with a flick/drag and gradually decelerate using friction; show spin time and optional challenge.

• Clickers and pop buttons should have satisfying pressed/released animation and optional generated clicks/pops. Sliders and dials should snap or glide according to the selected object.

• Use haptics on supported devices for clicks, detents and impacts. Offer haptics strength presets only if the device API can support a meaningful distinction; otherwise use simple on/off.

• Allow switching objects without leaving the experience. Keep sound optional and avoid repetitive loud loops.

K. Liquid Playground

• A fluid-like interactive canvas. User can stir, drag, drop objects, create ripples and observe the liquid react.

• Liquid settings: colour, viscosity (thin/thick), density, surface tension/intensity, transparency and flow speed. Include a few clearly explained presets.

• Material behaviours: miscible colours blend; immiscible liquids form separate regions; corrosive mode visually reacts with selected safe abstract materials; do not imply real chemistry.

• Interactive objects: sphere, cube, paddle, dropper, divider, floating object and absorbent sponge. Objects should produce distinct displacement, ripples, sinking/floating or mixing effects.

• Use a performant 2D approximation first. Do not attempt expensive full fluid dynamics if it compromises mobile performance. A visually convincing particle/grid-based simulation is acceptable.

• Sound should respond to stirring, splashes and object interactions. Haptics should be optional and capability-gated.

• Add reset, clear, pause, presets and a help overlay describing each object/material.

6. Navigation and visual design system

• **Design direction:** premium dark base with warm neutral text, restrained accent colours, subtle borders, soft depth and crisp typography. Avoid generic dashboard clutter and excessive gradients/glows.

• **Pixel identity:** reserve pixel fonts, pixel borders, limited palettes and dithering for Garage and selected arcade/game details. Do not force pixel styling onto every module.

• **Navigation:** home cards should be discoverable and visually distinct. Search/filter by Play, Create, Listen, Relax and Experiment.

• **Responsive:** desktop may use side navigation or a wide grid; mobile should use a compact header and single-column or two-column cards. Game controls must remain reachable.

• **Motion:** use motion to explain cause and effect, not decoration. Respect prefers-reduced-motion and provide a low-motion setting.

• **Consistency:** same button hierarchy, focus ring, modal style, toast style, tooltips, loading states and reset confirmation patterns.

7. Suggested architecture

Build each experience as an isolated module with a shared shell and shared services. Keep simulation logic separate from rendering and UI state.

• **App shell:** routing/navigation, home screen, global settings and shared design tokens.

• **Audio service:** one managed AudioContext, source routing, per-experience nodes, master gain, sound effects and cleanup.

• **Haptics service:** feature detection, user preference, vibration patterns and graceful fallback.

• **Storage service:** versioned local settings, scores, presets and art metadata; handle malformed or unavailable storage safely.

• **Canvas utilities:** resize handling, device-pixel-ratio scaling, animation lifecycle, pointer mapping and performance quality controls.

• **Game logic:** pure/testable state updates separated from UI and animation loops.

• **Asset policy:** original/generated pixel art and sound, or assets with compatible licences. Keep assets small and document licences.

Suggested project structure

```
src/ app/ # routing, shell, home, settings components/ # buttons, knobs, modals, cards, gauges
design/ # tokens, typography, themes services/ # audio, haptics, storage, capabilities experiences/
audio-lab/ garage/ generative-art/ atmosphere/ color-clash/ lights-out/ number-merge/
newtons-playground/ kinetic-sand/ fidget-lab/ liquid-playground/ hooks/ utils/ public/ icons/
sounds/ manifest.webmanifest service-worker.js
```

8. Build order and scope control

Do not try to build all eleven experiences at full fidelity in one pass. First create a stable shell and implement vertical slices. Each slice should be playable and testable before moving on.

• **Phase 1 — Foundation:** app shell, responsive home, design system, routing, settings, audio/haptics capability services, local persistence and GitHub Pages deployment.

• **Phase 2 — Quick wins:** Lights Out, Number Merge and Color Clash. These validate navigation, game state, score persistence and touch controls.

• **Phase 3 — Tactile modules:** Fidget Lab, Newton's Playground and Kinetic Sand. Validate canvas lifecycle, physics and optional haptics.

• **Phase 4 — Audio:** Audio Visualizer/Output Lab and Atmosphere. Validate audio graph, local file handling and sound lifecycle.

• **Phase 5 — Signature modules:** Garage Simulator and Liquid Playground. These have the most bespoke interaction and simulation work.

• **Phase 6 — Creative tools:** Generative Art Studio, photo import, geometric conversion, recolouring and export.

• **Phase 7 — Polish:** PWA/offline testing, mobile performance, accessibility, browser compatibility, onboarding and final visual consistency.

Minimum viable rule: every card shown on the home screen must open a working experience. If a module is not implemented yet, mark it visibly as Coming Soon and do not present fake controls as functional.

9. Quality assurance checklist

• All eleven routes load directly and through in-app navigation; browser refresh does not break nested routes.

• GitHub Pages deployment works from the repository's project path, including assets and service worker scope.

• No console errors, broken images, missing fonts or unhandled promise rejections during normal use.

• Audio starts only after a user gesture; mute and volume settings work; AudioContext is resumed correctly when needed.

• Audio nodes, animation frames, event listeners and timers are cleaned up when leaving an experience.

• Haptics are capability-detected and never required to complete an action.

• Touch and keyboard controls work; controls do not cause accidental page scrolling during active gameplay.

• Local data survives reload; reset works and is clearly confirmed.

• Large canvases adapt to resize and device pixel ratio; inactive experiences stop rendering.

• Reduced motion, sound off, haptics off and low-performance settings remain usable.

• Photo processing remains local and export produces a valid image.

• PWA install and offline app-shell behaviour are tested on supported browsers; limitations are documented.

10. Master prompt for the coding agent

Copy the prompt below into your coding agent after attaching this specification. It is intentionally strict about planning, architecture and not pretending incomplete features work.

```
You are a senior frontend engineer, interaction designer and browser-game developer. Build the product described in the attached document, “THE PLAYGROUND — Product specification & master build prompt”. Treat the document as the source of truth. The product is a multifunctional static web app intended for GitHub Pages. Use a modular architecture and a polished, responsive interface. Prefer Vite + React + TypeScript unless the existing repository already has a sound equivalent setup. Use Canvas 2D and Web Audio API where appropriate. BEFORE CODING: 1. Inspect the entire existing repository and identify its framework, scripts, deployment setup and current features. Do not overwrite or restructure working code blindly. 2. Report the proposed architecture, implementation phases, dependencies and any browser capability limitations. 3. Identify the GitHub Pages base path and ensure routing/assets/service-worker scope will work under a repository subpath. 4. Create a concise implementation checklist and then begin with Phase 1. IMPLEMENTATION RULES: - Build the shared app shell, navigation, design tokens, settings, local persistence, audio service and haptics service before implementing all experiences. - Keep each experience isolated and reusable. Separate simulation/game logic from rendering and UI. - Do not create non-functional placeholder controls.
```

```
If a feature is not implemented, label it Coming Soon or leave it out of the active UI. - Do not claim global system audio processing. The audio EQ must process only audio sources routed through this app's Web Audio graph. - Use navigator.vibrate only when supported and after user interaction. Provide a graceful fallback and a settings toggle. - Never autoplay audio. Respect browser permissions and user preferences. - Process uploaded photos locally in the browser. Do not upload user images. - Use original or properly licensed art and sound assets. Do not copy proprietary game visuals, music or car branding. - Optimize Canvas simulations for mobile. Pause inactive scenes and clean up resources on route changes. - Support touch, mouse and keyboard. Include reduced-motion and low-performance options. - Persist settings, scores and presets locally. Version stored data and handle invalid storage safely. - Make the Garage's Arcade Drive and Engine Lab two genuinely distinct modes, as specified. - In Engine Lab, keep RPM gauge on the left, car display in the centre and speed gauge on the right. The gauges are mandatory in this mode. - In Arcade Drive, prioritize lane switching and obstacle avoidance; speed controls are optional, and do not show misleading RPM/gear telemetry if not simulated. - Keep the Garage pixel-art style distinctive. Do not force pixel styling onto every other module. WORKFLOW: - Implement one phase at a time. After each phase, run the available build, lint and tests; fix errors before continuing. - Keep the app deployable at every phase. - After each phase, summarize files changed, features completed, tests run, known limitations and the next phase. - Do not stop after producing a plan. Continue implementing unless blocked by a genuine missing requirement. - Do not silently remove existing user functionality. - Before finishing, test the production build and GitHub Pages deployment path, verify all navigation and module entry points, and provide exact deployment instructions. DEFINITION OF DONE: The app has a coherent home screen and shared settings; each implemented experience works end-to-end; controls provide immediate feedback; sound/haptics degrade gracefully; local data persists; mobile and desktop layouts are usable; no visible control falsely implies functionality; and the production build is suitable for GitHub Pages.
```

11. Product decisions to keep in mind

• The requested audio visualizer is an in-app audio processor, not a system-wide equalizer.

• Haptics are an enhancement, not a requirement. Many desktop browsers and some mobile browser combinations will not expose vibration.

• The Garage's engine and drivetrain can be an intentionally simplified model. A convincing sound/needle response is more important than a full vehicle simulation.

• Photo-to-geometric-art conversion should begin with a performant approximation (colour quantization plus geometric segmentation/facets), then improve only if performance allows.

• Liquid simulation, kinetic sand and Newtonian physics are the most performance-sensitive modules. Start with stable 2D approximations and tune for mobile.

• The app is large. Deliver a stable, coherent product incrementally rather than generating eleven shallow mockups at once.

End of specification.